CSA17 Assignment 2 – Answers

Task 1 – Uniform-cost, Greedy and A* on different maps

A* with Manhattan distance is the right choice for a fleet that drives 95% of its trips on open roads: it is always optimal and expands about 27 times fewer nodes than UCS.

Heuristic and proof

$h(n) = |r(n) - r(goal)| + |c(n) - c(goal)|$. It is the exact cost of the relaxed problem with all obstacles removed. Every real route is also legal in the relaxed problem, so $h \le$ true cost: **admissible**. One step changes h by exactly 1, so $h(n) \le 1 + h(n')$: **consistent**. A* therefore never re-opens a node and its first goal expansion is optimal.

Results (100×100 grids, unit step cost)

Map	Algorithm	Expanded	Generated	Max frontier	Cost (optimal)
A: open, 5% obstacles	UCS	9,487	9,491	104	190 (190)
	Greedy	199	549	351	196 (190)
	A*	339	683	345	190 (190)
B: U-shaped trap	UCS	9,789	9,807	121	142 (142)
	Greedy	2,881	3,091	211	142 (142)
	A*	3,583	3,793	211	142 (142)
C: perfect maze	UCS	4,791	4,794	9	2,168 (2,168)
	Greedy	3,510	3,533	24	2,168 (2,168)
	A*	4,614	4,619	10	2,168 (2,168)
D: goal sealed off	UCS	9,951	9,951	111	no path
	Greedy	9,951	9,951	320	no path
	A*	9,951	9,951	322	no path

Predictions against what happened

- **Open map:** UCS ≫ A* > Greedy in expansions. Held.
- **U-trap:** I predicted Greedy would return a sub-optimal path. Wrong: it found the optimal 142, but needed 2,881 expansions, 14 times its open-map count, and A* expanded more than Greedy because it must also prove optimality.
- **Maze:** all optimal, nearly equal effort, because Manhattan distance is far below the true cost. Held.

- **Sealed goal:** identical exhaustive expansion. Held.

200 random open maps (5% obstacles)

Algorithm	Mean expanded	Sub-optimal runs	Mean excess cost
UCS	9,296	0 / 200	0%
Greedy	167	195 / 200	+2.8% (max +8.7%)
A*	339	0 / 200	0%

Greedy's excess cost grows with clutter, from 3% at 5% obstacles to 21% at 30%, while A* stays optimal.

Recommendation

A*. Greedy saves well under a millisecond per trip but drives a longer route on almost every trip and loses its guarantee on cluttered trips. Choose Greedy only if any feasible route is enough and time is hard-limited; choose UCS when no useful heuristic exists (maze-like layouts, where A* gains only 4%).

Task 2 – Online search agent

Online DFS is only reasonable for one or two missions in a small unmapped facility; with stable maps and repeated missions, explore once and then plan on the stored map.

Agents tested

Online DFS moves to an unvisited neighbour and physically walks back when stuck (backtracking counts as moves). LRTA* keeps a table H(cell), starting from Manhattan distance, and updates H(s) = min over neighbours of (1 + H(n')). Mazes of 17, 33 and 65 cells per side, with 0%, 15% and 40% of inner walls removed (dead-end density 0.07 down to 0.01); 20 mazes per configuration.

Results

Maze	Loops	Optimal (far)	CR near	CR mid	CR far	Max CR far
17×17	0	93			1.4	1.7
17×17	0.15	44			3.0	5.5
17×17	0.4	31			3.7	8.3
33×33	0	305			1.8	3.0
33×33	0.15	81			6.6	12.1
33×33	0.4	62			9.0	16.5
65×65	0	1,035	4.5	2.3	1.8	2.9
65×65	0.15	159	29.0	20.7	15.4	25.4
65×65	0.4	126	50.9	21.4	20.0	35.4

CR = moves ÷ optimal cost with a full map. Near and mid targets were run for every size; the table shows them for 65×65 only.

- **Unreachable target:** moves = 2 × (reachable cells − 1) exactly, in every configuration. Failure is declared only after the whole component is walked twice (4,380 moves on a 65×65 maze with 15% loops).
- **Distance:** CR is largest and most variable for near targets (maximum 149 on 65×65).
- **Surprise:** fewer dead ends (more loops) gave a higher CR, because loops shorten the optimal route while DFS still wanders.
- **LRTA*:** trial 1 costs 3.5 to 15 times more than online DFS (1,009 vs 130 moves on 17×17 perfect; 9,297 vs 2,662 on 65×65 with loops). It converges to the optimal cost in 18 to 24 trials (17×17), 75 to 117 (33×33) and about 350 (65×65).

Operating boundary

Use online DFS when no map exists, the facility is small enough that exploring it fully (2 × cells moves) is affordable, the target is expected to exist, and there are at most about 2 to 3 missions (break-even 1.9 to 3.1 missions). Stop recommending it when typical CR exceeds about 5 to 10 (33×33 and 65×65 mazes with loops), or missions repeat.

Changed condition: 20 rovers, 20 facilities, maps stable for months

The recommendation changes. Use online search only for the first mission in each facility, store the map, then plan with A* or UCS. For a 65×65 facility and 100 missions: about 240,700 moves repeating online DFS against about 20,700 for map-once-then-optimal (11.6 times less). LRTA* is not the right learner here: its first trial is expensive and it needs 18 to 350 trials, whereas an explicit map is optimal from mission 2.

Task 3 – Alpha-beta pruning

The same tree needed 127 or 3,622 leaves depending only on move order, so b and d alone do not determine alpha-beta's cost.

Set-up

Uniform tree, b = 4, d = 6 (4,096 leaves), distinct leaf values, root = MAX. Trees 1 and 2 have identical leaves; only child order differs (best-first, worst-first). Test 3a: random order over 200 trees. Test 3b: all leaves equal. Alpha-beta's root value was checked equal to minimax in every run.

Predictions and results

Test	Predicted leaves	Observed leaves (nodes, cutoffs)	Verdict
Minimax	4,096	4,096 (5,461)	reference
1. Best-first order	127 ($b^{\lceil d/2 \rceil} + b^{\lfloor d/2 \rfloor} - 1$)	127 (268, 99)	exact match

Test	Predicted leaves	Observed leaves (nodes, cutoffs)	Verdict
2. Worst-first order	4,096, no cutoffs	3,622 (4,925, 245)	wrong
3a. Random order (mean of 200)	about 512 (range 256 to 1,024)	904	inside range, 1.8× above estimate
3b. All leaves equal	127	127 (268, 99)	exact match

Why

A subtree is skipped only when an already-explored sibling or ancestor gives a bound that refutes it. With best-first order the first child sets the tightest bound at once, so every later sibling is cut after one child. With worst-first order the bound is loose when each sibling is searched.

The worst-first prediction failed because bounds are also inherited from ancestors: cutoffs are zero for d ≤ 3 but appear from d = 4 (4 cutoffs at d = 4, 240 at d = 6 in the growth runs). Equal leaves behave like perfect ordering because the cutoff test β ≤ α fires on ties.

With the same b and d the leaves evaluated ranged from 127 to 3,622 (28.5×) and the effective branching factor from 2.24 to 3.92. O(b^d) and O(b^(d/2)) only bound a family of trees; a real engine's cost depends on its move ordering.

Task 4 – Depth-limited minimax

Plain minimax cost is b^d per move; on Connect-4 (7 columns) it is practical to depth 5 and too slow at depth 6 under a 1 s per move limit.

Game and evaluation function

Connect-4 on cols × rows boards, b = number of columns, MAX = player 1. A win returns ±(10^6 − ply), a draw 0. At the depth cut-off the position is scored by counting open 4-windows (69 on 7×6): a window with only one colour's pieces scores +/− 1, 5, 25 or 1,000 for 1, 2, 3 or 4 pieces; windows with both colours score 0. One evaluation takes 12.7 µs.

Derivation

$$N(d)=1+b+\dots+b^{d}=\frac{b^{d+1}-1}{b-1}=\Theta(b^{d}),\qquad E(d)=\frac{b^{d}-1}{b-1},\qquad V(d)=b^{d}$$

Level i has b^i nodes; N is all nodes examined, E the nodes expanded, V the evaluation calls at the cut-off depth. The leaf share tends to (b − 1)/b, which is 86% for b = 7, and cost grows by a factor b per extra ply. The formulas are exact while no win and no full column is possible within d plies (d ≤ 6 on 7×6; d ≤ 5 on 5×5 and 6×5).

Verification (empty 7×6 board)

d	Nodes (formula = observed)	Evaluations (formula = observed)
1	8	7

d	Nodes (formula = observed)	Evaluations (formula = observed)
2	57	49
3	400	343
4	2,801	2,401
5	19,608	16,807
6	137,257	117,649

All 16 depth and board combinations (5×5, 6×5, 7×6) matched the formulas exactly. On mid-game positions the tree is smaller (68% of the formula at d = 6: 93,383 nodes) because wins and full columns stop expansion.

Time per move (mid-game 7×6, 8 positions; 5 at d = 6)

d	Mean nodes	Mean time (s)	Max time (s)
4	2,340	0.034	0.046
5	15,683	0.200	0.235
6	93,383	1.148	1.509

Acceptable response time: 1 second per move (turn-based interactive play). Plain minimax is practical up to depth 5 and impractical from depth 6. Time grows 5.7 to 7 times per ply, so depth 7 would need about 7 s and depth 8 about 42 s (extrapolation, not measured).

Quality of the move

Opponent	Depth 1	Depth 2	Depth 3	Depth 4
Random (20 games)	20 W	20 W	20 W	20 W
Depth-1 player (20 games)	n/a	10 W, 10 D	20 W	17 W, 3 L

I predicted at least 70% wins at depth 1 and at least 90% from depth 3, which was too cautious: all depths won 20/20 against random, so that metric cannot separate depths. Depth 4 losing 3 games to depth 1 shows a deeper search is not always better with a hand-made evaluation.

Where the inefficiency comes from

72% of nodes (mid-game) are leaf evaluations, and evaluation takes 74% of the time at d = 5. Alpha-beta needs only 1,688 of 83,863 evaluations at d = 6 (2%) to reach the same value, and at d = 6 there are 86,794 leaf visits but only 10,803 distinct positions. A better strategy must skip moves that cannot change the decision, try promising moves first, and reuse results for repeated positions.

Task 5 – Constraint propagation for exam timetabling

Propagation alone solved one timetable and detected failure in another with zero search, but it is sound and not complete, so it must be combined with search.

Procedure derived from the principle

- **Representation:** exams are variables, slots are domain values. Each conflict gives two directed arcs (X,Y) and (Y,X) with the predicate "slots differ".
- **REVISE(X,Y):** remove every value of X that has no compatible value in D(Y); each compatibility test is one constraint check.
- **After D(X) changes:** only arcs (Z,X) for the other neighbours Z ≠ Y must be re-examined, because they used D(X) as support. A queue holds pending arcs.
- **Wipe-out:** an empty domain means no solution.
- **Search variants:** BT (no propagation), BT+AC (propagate once, then BT), MAC (propagate after each assignment, undo on backtrack).

Tests

Instance	Method	Result	Nodes	Backtracks	Constraint checks
A: 8 exams, short domains	BT	solved	79	33	96
	BT+AC and MAC	solved by propagation alone	0	0	36 (8 removals)
B: A plus one conflict	BT	no solution	65	32	82
	BT+AC and MAC	wipe-out detected	0	0	36 (9 removals)
C1: K4 with 3 slots	BT	no solution	48	16	75
	BT+AC	no solution	48	16	123 (0 removals)
	MAC	no solution	9	4	171
C2: 30 exams, 3 slots, 10% fixed	BT	solved	153	35	295
	BT+AC	solved	48	0	580
	MAC	solved	30	0	847

On C1 propagation removes nothing (every value has support), so BT+AC only adds work; MAC needs search but visits far fewer nodes. Propagation reduces nodes, not necessarily constraint checks: on C2 plain BT used the fewest checks.

Growth with the number of exams (3 slots, 4.4 conflicts per exam, 10% fixed, 15 instances per size)

Exams	BT median nodes	BT+AC median nodes	MAC nodes	BT mean checks	MAC mean checks
10	28	17	10	60	282
20	74	37	20	308	614
30	263	125	30	1,512	932
40	3,304	264	40	8,863	1,277
50	11,620	466	50	40,131	1,529
60	14,223 (2 runs hit the 200,000-node limit)	1,576	60	78,450	2,035

BT grows steeply; MAC visits exactly n nodes with no backtracking and its checks grow about 7 times against about 1,300 times for BT. At small n MAC does more checks than BT (crossover between 20 and 30 exams). These planted instances are fairly easy, so they do not show that MAC never backtracks; C1 shows it does when the instance is unsatisfiable.

Why propagation is correct

Invariant: every solution of the original problem uses only values still in the current domains. Suppose a is removed from D(X) because no value in D(Y) is compatible with it, and some solution S had S(X) = a. Then S(Y) is compatible with a, and by the invariant S(Y) is in D(Y), contradicting "no support". So no removal loses a solution, the reduced problem has exactly the same solutions, and an empty domain proves there is none. Each removal deletes one of finitely many values, so the procedure terminates. The converse fails: K4 with 3 slots has no empty domain yet no solution.

Task 6 – Courier tour: exhaustive versus local search

Exhaustive search is practical up to 9 customers under a 2 s limit; from 10 customers use 2-opt with 10 random restarts.

Exhaustive search design

City 0 is the depot; a tour is a permutation of customers 1 to n. Depth-first construction with a used flag, so no permutation repeats. Distances are symmetric, so a tour and its reverse are the same: only permutations with first customer < last customer are evaluated, giving exactly **n!/2 tours**. Length is accumulated incrementally; a single best-so-far is kept; no pruning.

Correctness checks

- Hand instance (depot (0,0); customers (2,0), (2,2), (0,2) and the centre (1,1)): optimum 8.82843 = 6 + 2√2, 12 tours examined (4!/2). By hand, the centre must be inserted into the perimeter of length 8 at minimum extra cost 2√2 − 2.

- Points on a circle (depot + 9 customers): exhaustive 309.0169943749 against the known polygon perimeter, error 1.1e-13.
- 40 random instances with n = 7 against an independent `itertools.permutations` brute force: 40 / 40 identical.

Growth

n	Tours (n!/2, observed = predicted)	Time (s)	Time ratio to previous n
8	20,160	0.032	5.3
9	181,440	0.354	11.1
10	1,814,400	3.49	9.9
11	19,958,400	37.4	10.7

The tour count matched n!/2 exactly for n = 4 to 11, and time grows by about n + 1 per added customer, as predicted. My formula for DFS nodes entered was wrong by exactly n!/2: the search still enters all n! leaf positions and only then rejects mirror images.

Why a few more customers cause a dramatic increase: a new customer can be inserted at any of n positions in every existing tour, so the tour count is multiplied by n + 1. At the measured 533,000 tours per second, n = 12 takes about 7.5 minutes, n = 15 about 14 days, n = 20 about 72,000 years (extrapolated). A machine 1,000 times faster buys only about 3 more customers.

Local search: 2-opt with random restarts

Start from a random tour; replace edges (a,b) and (c,d) by (a,c) and (b,d) when that shortens the tour; stop at a local minimum; repeat from R random starts and keep the best. 20 runs per row.

n	Restarts	Reference	Mean excess	Runs reaching reference	Time per run (ms)
8	1 / 10	exhaustive optimum	0% / 0%	20 / 20 and 20 / 20	0.015 / 0.135
10	1	exhaustive optimum	1.84% (max 16.5%)	14 / 20	0.022
10	10	exhaustive optimum	0%	20 / 20	0.212
11	1	exhaustive optimum	2.38% (max 6.0%)	10 / 20	0.026
11	10	exhaustive optimum	0%	20 / 20	0.265
20	10	best of 20 runs (not proven optimal)	0.26%	14 / 20	0.92
50	10	best of 20 runs	1.46%	2 / 20	6.7

n	Restarts	Reference	Mean excess	Runs reaching reference	Time per run (ms)
100	5	best of 20 runs	2.72%	1 / 20	12.0

Why local search can fail: a tour with no improving 2-opt neighbour is a local minimum. From 20,000 random starts, the n = 10 instance has 3 local minima (63.1% of starts reach the optimum; worst minimum 16.5% above it) and the n = 11 instance has 6 (43.1%; worst 6.0%). On the n = 8 instance every start reaches the optimum, so small instances can succeed trivially. Plateaus (equal-length neighbouring tours) would stop a strict-improvement search as well; my random instances have no exact ties, so this part is explained, not measured. Random restarts fixed n = 10 and 11 (20/20).

Deployment decision

Response-time limit: 2 seconds (planning when a dispatcher changes a delivery). Exhaustive search takes 0.35 s at n = 9 and 3.5 s at n = 10, so use it for **n ≤ 9** and **2-opt with 10 restarts for n ≥ 10**. For n ≥ 20 the tours are within about 0.3 to 3% of the best found in 20 runs, which is not a proven optimum, so the true gap may be larger. A compiled implementation would move the switch point only to about n = 11 or 12. Times are for Python on one core.

Task 7 – Day-4 design challenge (redesign of the Task 4 minimax)

Alpha-beta with centre-first ordering and iterative deepening gives the same move values as minimax with about 50 times fewer evaluations at depth 6, so it searches three plies deeper within 1 s.

A. Diagnosis

- **Where the effort goes:** evaluation calls. At depth 5, 72% of nodes are evaluations and evaluation takes 74% of run time (12.7 µs per call).
- **What grows:** evaluations = b^d; time per move grows 5.7 to 7 times per ply.
- **Repeated work:** at depth 6 there are 86,794 leaf visits but only 10,803 distinct positions (8×); at depth 5, 12,808 against 3,022. Alpha-beta needs only 2% of the evaluations to find the same value.
- **Cause:** every branch gets equal effort, no bounds from explored siblings are used, children are tried in a fixed order, and nothing is reused between positions or depths.

B. Objectives

1. **O1:** same move values as minimax at the same depth.
2. **O2:** at least 10× fewer evaluation calls at depth 6.
3. **O3:** at most 1 s per move at depth 7 or more.
4. **O4:** small extra memory (a table of about the expanded positions).

C. Design

A subtree matters only if its value can fall inside the window (α, β) of what each player can already guarantee, so subtrees outside it are skipped (cutoff when α ≥ β). Task 3 showed the saving depends on trying the best move first, so the order is the second ingredient: static centre-first order (central columns lie in more windows), and iterative deepening with a table that stores, per position, the best

move from the previous depth and tries it first. Keying the table by position also shares ordering across transpositions. Costs: repeated shallow iterations, table memory, hashing overhead, and dependence on ordering quality.

D. Predicted evaluation calls at depth 6 (before implementing)

Variant	Predicted
V0 plain minimax	117,649
V1 alpha-beta, natural order	about 11,400
V2 alpha-beta, centre-first	about 3,000
V3 iterative deepening with move table (all iterations)	about 1,500

E. Results (evaluation calls; mid-game values are means over positions)

Position, depth	V0	V1	V2	V3
Empty board, d = 4	2,401	722	142	240
Empty board, d = 5	16,807	2,427	556	805
Empty board, d = 6	117,649	11,776	875	1,784
Mid-game, d = 4	1,921	810	526	210
Mid-game, d = 5	12,647	3,180	1,555	684
Mid-game, d = 6	83,863	17,506	11,502	1,688

Seconds per move at mid-game d = 6: V0 1.453, V1 0.307, V2 0.197, V3 0.030 (48× faster). V3 at depth 7 takes 0.081 s on average (max 0.157 s) and at depth 8 0.137 s (max 0.221 s). **O1:** the root values were identical for all four variants on every test position, and the root-move value vectors of V3 and minimax at depth 5 were identical on 30 of 30 random positions.

Games (20 each, colours alternated)	Result for the redesign	Seconds per move (redesign / minimax)
V3 depth 5 vs minimax depth 5	10 W, 10 L	0.017 / 0.115
V3 depth 7 vs minimax depth 5	19 W, 1 L	0.122 / 0.120
V3 depth 6 vs minimax depth 5	8 W, 12 L	0.059 / 0.116
V3 depth 8 vs minimax depth 5	11 W, 5 D, 4 L	0.474 / 0.138

F. Defence

- **Objectives met:** O1 yes; O2 yes (117,649 to 1,784 on the empty board, 66×; 83,863 to 1,688 mid-game, about 50×); O3 yes (depth 8 in at most 0.22 s); O4 holds as a bound only (one table entry per expanded position; I did not measure bytes).
- **Predictions:** V1 (about 11,400) and V3 (about 1,500) were close (11,776; 1,784 and 1,688). V2 was predicted badly: 875 on the empty board but 11,502 mid-game, because static ordering depends on the position. This is why the adaptive ordering of V3 pays off mid-game (6.8× fewer evaluations than V2) even though it costs more than V2 on the empty board (1,784 against 875).
- **Quality:** at the same depth the redesign plays identically (10 to 10); the gain is depth for the same time. At equal thinking time depth 7 beat depth-5 minimax 19 to 1.
- **A result that did not go as hoped:** depth 6 (8 to 12) and depth 8 (11 W, 5 D, 4 L) did not beat depth-5 minimax. A plausible cause is an odd/even depth effect of my evaluation function; I did not test it. The redesign cannot fix the evaluation function.
- **When the original is still fine:** depth 3 or less, when exact values of every root move are needed (depth 8 took 0.47 s per move with a full window per root move), or as the reference implementation. With adversarial ordering alpha-beta approaches minimax's cost (Task 3: 88% of the leaves).